The Production Incident Starter Kit for Engineers

A Practical Field Guide for Debugging Production Failures Faster

---

Introduction

Production incidents are an inevitable part of modern software systems.

Even the most mature engineering teams encounter outages, degraded services, and mysterious failures in production environments. What separates experienced engineers from struggling teams is not whether incidents occur — but **how quickly they can understand and resolve them**.

In many organisations, incident response still looks like this:

1. Alerts fire across monitoring tools
2. Engineers jump into dashboards and logs
3. Multiple theories emerge
4. The team spends valuable time testing the wrong hypotheses

The first **20–30 minutes of an incident** are often spent simply trying to determine **where to start**.

This starter kit exists to solve that exact problem.

It provides practical tools used by experienced SREs, DevOps engineers, and backend developers to quickly move from:

Symptoms → Signals → Root Cause

Inside this starter kit you will find three practical resources:

1 An **Incident Debugging Cheat Sheet**
2 A **Mini Edition of AWS Log Search Recipes**
3 A **Production Incident Post-mortem Template**

Each resource is designed to be used **immediately during real production incidents**.

---

1. Incident Debugging Cheat Sheet

One of the most common mistakes during incident response is treating the **first visible symptom as the root cause**.

For example:

- A user sees a **502 Gateway Error**
- Engineers assume the **API gateway is broken**
- Hours later they discover the issue was actually a **downstream service crash**

Experienced incident responders begin by asking a simple question:

"What type of failure pattern does this symptom resemble?"

The cheat sheet below provides common production symptoms, their typical causes, and the first place engineers should investigate.

Incident Debugging Cheat Sheet

Symptom	Likely Cause	First Check
502 Gateway Error	Downstream service crash	Service logs
504 Gateway Timeout	Slow backend dependency	Dependency metrics
Connection Timeout	Database pool exhaustion	Database metrics
Authentication Failure	Expired token or auth outage	Auth service logs
Sudden latency spike	Resource saturation	CPU / memory metrics
Intermittent failures	Load balancer imbalance	Load balancer logs
Requests stuck in queue	Worker thread starvation	Worker service metrics
Increased error rate	Deployment regression	Recent deployment history
Cache miss storm	Cache eviction or restart	Cache metrics
API returning partial data	Dependency degradation	Upstream service logs

---

The 5 Questions Every Incident Responder Should Ask

Before diving into logs or dashboards, experienced responders typically ask five simple questions:

1 What changed?

Many incidents are triggered by change.

Check:

- Recent deployments
- Infrastructure updates
- Configuration changes
- Dependency updates

If something changed shortly before the incident started, it is often the **primary suspect**.

---

2 What dependencies are common across the failures?

If multiple services are failing, they may share a dependency such as:

- A database
- A message queue
- A cache
- An external API

Identifying shared dependencies quickly can dramatically narrow down the investigation.

---

3 What is still working?

Understanding what remains healthy helps eliminate possibilities.

For example:

- If read queries work but write queries fail → database replication issue
- If internal calls work but external traffic fails → network edge problem

Healthy systems provide valuable clues.

---

4 Did rollback make things better or worse?

Rollback is a powerful diagnostic tool.

If rollback improves the system:

The incident likely relates to the deployment.

If rollback changes nothing:

The root cause may be elsewhere — infrastructure, dependencies, or configuration.

---

5 Could the first signal be misleading?

One of the biggest mistakes in incident response is assuming the **first error message explains the problem**.

Often the first signal is just **the first place the system noticed the failure**, not the actual cause.

For example:

A timeout error in an API may actually be caused by:

- database locking
- queue backlog
- thread pool exhaustion

The symptom is rarely the root cause.

---

2. AWS Log Search Recipes (Mini Edition)

Logs remain one of the most valuable sources of truth during incidents.

While dashboards show aggregated signals, logs often reveal the **exact sequence of events** leading to a failure.

However, many engineers struggle to search logs effectively under pressure.

This section provides practical **CloudWatch Logs Insights queries** engineers can run immediately during incidents.

Recipe 1 — Find Timeout Errors

```
fields @timestamp, @message
| filter @message like /timeout/
| sort @timestamp desc
| limit 50
```

Use this when systems are experiencing:

- connection timeouts
- gateway timeouts
- dependency delays

This query surfaces the most recent timeout events across logs.

Recipe 2 — Identify Error Spikes

```
fields @timestamp, @message
| filter @message like /ERROR/
| stats count() by bin(1m)
```

This query groups error events per minute.

It helps answer the question:

“When exactly did the incident start?”

Recipe 3 — Find Failing Services

```
fields @timestamp, service, @message
| filter @message like /exception/
| stats count() by service
| sort count desc
```

This query identifies which services are generating the most exceptions.

It helps pinpoint **the most likely failing component**.

Recipe 4 — Identify Slow Requests

```
fields @timestamp, latency, endpoint
| filter latency > 2000
| sort latency desc
| limit 20
```

Useful during performance incidents.

It surfaces the slowest requests across services.

Recipe 5 — Investigate Deployment Issues

```
fields @timestamp, @message
| filter @message like /deployment/
| sort @timestamp desc
| limit 20
```

This helps correlate incidents with deployment activity.

Many outages are introduced shortly after deployments.

Why Log Search Skills Matter

Even with advanced observability platforms like:

- Datadog
- Splunk
- CloudWatch
- Grafana

engineers still rely heavily on logs to answer questions such as:

- What actually happened?
- Which request failed first?
- Which dependency caused the error?

Log analysis remains a **core skill for incident responders**.

---

3. Incident Post-mortem Template

Resolving an incident is only half the work.

High-performing engineering teams invest time in **learning from failures** so that the same incident does not happen again.

This process is typically called a **post-mortem**.

A good post-mortem focuses on learning rather than blame.

The goal is not to identify who caused the incident, but to understand:

- What happened
- Why it happened
- How it can be prevented in the future

Incident Post-mortem Template

Incident Summary

Provide a brief overview of the incident.

Example:

On 12 March at 14:03 UTC, the payments API experienced increased latency due to database connection pool exhaustion. The issue lasted 23 minutes and impacted approximately 18% of customer transactions.

Incident Timeline

A timeline helps reconstruct events.

Example:

Time	Event
14:03	Error rate begins increasing
14:05	Alert triggered
14:07	Engineers join incident bridge
14:12	Database connection pool saturation discovered
14:18	Temporary scaling applied
14:26	System stabilised

Timelines reveal how incidents unfold.

Root Cause

This section explains the underlying cause of the failure.

Example:

A recent deployment increased database query concurrency but the connection pool configuration was not updated accordingly, resulting in pool exhaustion.

Contributing Factors

Rarely does a single issue cause an incident.

Often several factors contribute, such as:

- missing alerts
- insufficient monitoring
- undocumented dependencies
- lack of capacity planning

Understanding these factors improves system resilience.

Preventative Actions

This section lists actions to prevent recurrence.

Examples:

- Increase database connection pool limits
- Add monitoring for connection saturation
- Introduce load testing before deployments
- Improve dependency documentation

Each action should have a **clear owner and timeline**.

Why This Starter Kit Matters

Modern systems are increasingly complex.

Applications depend on:

- microservices
- distributed databases
- external APIs
- messaging platforms
- cloud infrastructure

When incidents occur, engineers are often overwhelmed by the volume of signals produced by monitoring tools.

This is why incident response requires structured thinking.

The resources in this starter kit help engineers:

- identify common failure patterns
- search logs effectively
- investigate incidents systematically
- learn from failures

These skills are essential for engineers working in:

- DevOps teams
- Site Reliability Engineering (SRE)
- Platform engineering
- Backend development

Final Thoughts

Production incidents will always occur.

But with the right approach, they can become powerful opportunities to improve systems.

The best engineering teams treat incidents not as failures, but as **valuable feedback loops**.

By improving incident investigation, debugging, and learning processes, teams become more resilient over time.

This starter kit is designed to help engineers take the first step towards **faster incident resolution and better system reliability**.

Want More Incident Engineering Resources?

If you found this guide helpful, you may also be interested in:

• Advanced AWS Log Search Recipes
• Production Failure Pattern Libraries
• Incident Engineering Frameworks
• ExplainError — structured failure interpretation for production systems